Clustering

Data Science

Clustering

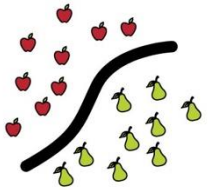
- k-means
- hierarchical clustering
- DBSCAN

MACHINE LEARNING

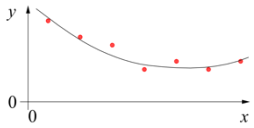
training target available (labeled or past data)

SUPERVISED

CLASSIFICATION



REGRESSION



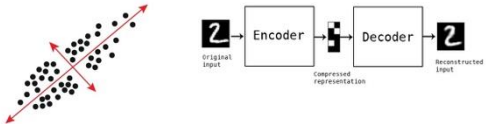
data not labeled in any way

UNSUPERVISED

CLUSTERING

SELF-SUPERVISED

DIMENSIONALITY REDUCTION



ASSOCIATION

no supervision, but goal-based interaction with environment

REINFORCEMENT LEARNING

LEARN STATE OR ACTION VALUES

LEARN POLICY DIRECTLY

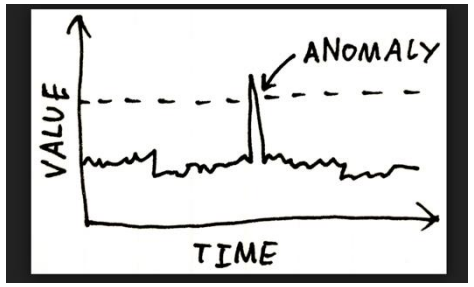
learning by teacher (high-dimensional curve fitting)

learning by observation (pattern recognition)

learning by trial-and-error (sequential decision making)

Use Cases for Unsupervised Learning

Anomaly Detection

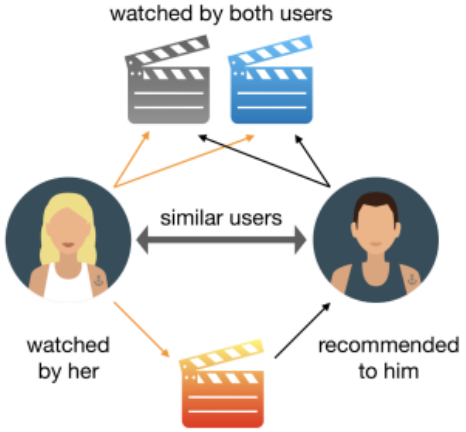


Fraud Detection

Risk Analysis

Time Series Reconstruction

Representation Learning



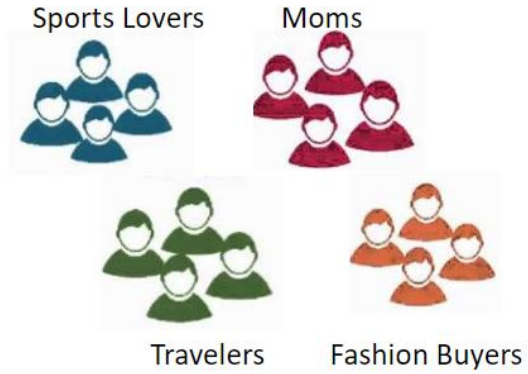
Recommender Systems

Image Compression

Drug Discovery

Topic Modeling

Structure Discovery



Customer or Market Segmentation

Social Network Analysis

Techniques: clustering, dimensionality reduction, association rules, …

What Is Clustering?

automatic grouping of similar objects into sets

(without predefined labels)

similar means similar in terms of the object's features

- customer segmentation
- document clustering
- image segmentation (regions with similar color, texture, or intensity)
- ...

Distance Metrics and Similarity

How to measure if points (in higher-dimensional space) are close?

- each feature corresponds to one dimension
- close means small differences across dimensions
  - Euclidean distance: $d_E(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{\sum_{i=1}^{d}(x_i - y_i)^2}$
  - Manhattan distance: $d_M(\boldsymbol{x}, \boldsymbol{y}) = \sum_{i=1}^{d}|x_i - y_i|$
- instead of distance, sometimes also similarity: higher = closer
  - cosine similarity: $\frac{\boldsymbol{x} \cdot \boldsymbol{y}}{\|\boldsymbol{x}\|\|\boldsymbol{y}\|}$
  - Pearson correlation: linear similarity between features

Preprocessing for Clustering

distance measures sensitive to scale

→ need for scaling

- **standardization**: $z = \frac{x-\mu}{\sigma}$
- **normalization**: scale all features to $[0,1]$

without scaling, distance dominated by features with larger numeric ranges

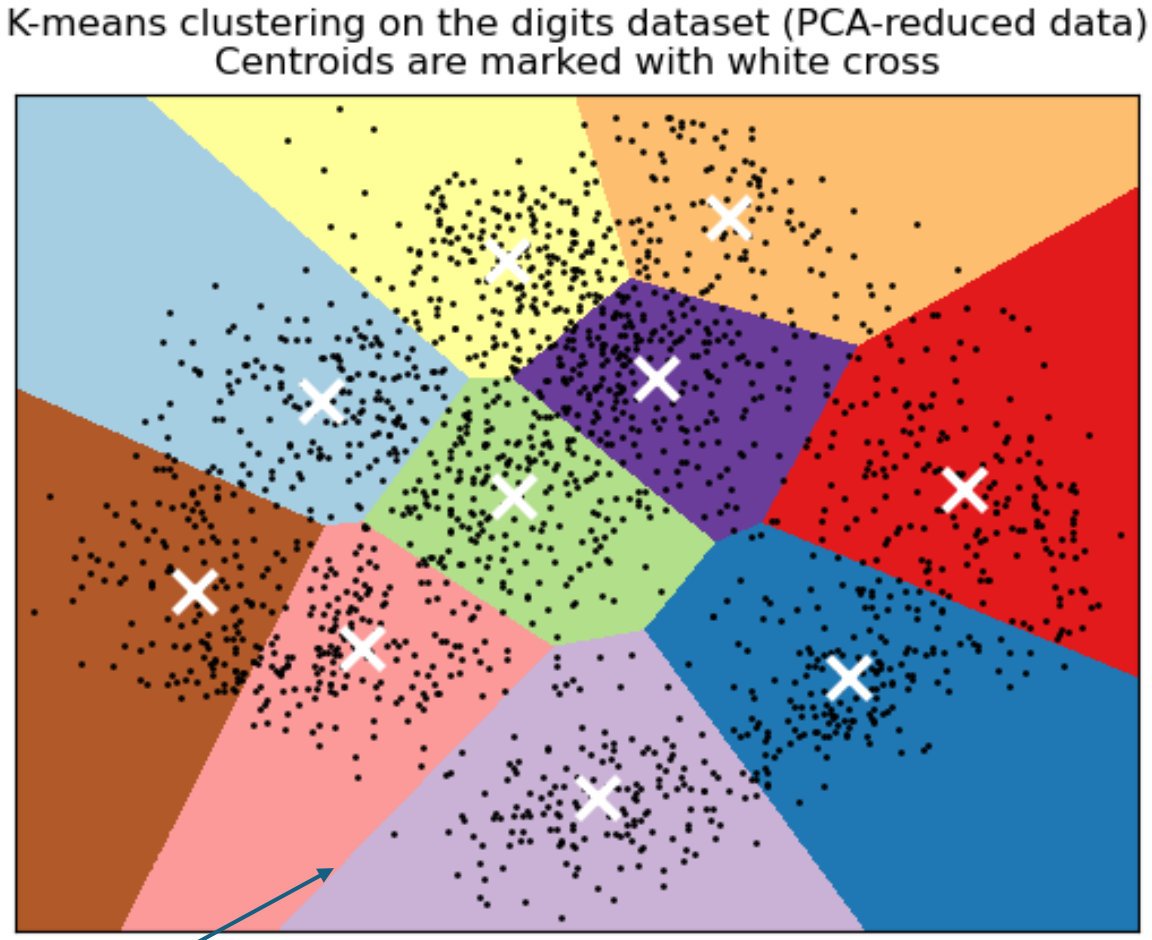
k-Means: Intuition

divide a dataset into k clusters

k is a hyperparameter (to be set before the algorithm runs)

each cluster is represented by its centroid (mean point)

each data point is assigned to the nearest centroid



decision boundaries (straight lines because k-Means was trained directly in the same 2-D space that is visualized)

k-Means Algorithm

Fit:

1. choose **k** (number of clusters)
2. initialize centroids: (randomly) place k centroids in data space
3. assign each point to the nearest centroid (according to computed **Euclidean distances**)
4. recalculate centroids of each cluster (**average** of all points assigned to the cluster)
5. iterate until convergence (centroids stop moving significantly or cluster assignments stop changing)

Predict:

For a new point: Assign it to the cluster with the **nearest centroid**

Objective Function

k-Means minimizes the sum of squared distances from points to their cluster center (mean):

$$\sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

Euclidean distances
→ spherical clusters

→ optimization probem

Manhattan distances (L1) → k-Medians (also recalculate the cluster centers using the median instead of the mean)

L2 → Mean, L1 → Median

L2 objective:

$$\min_a \sum_{i=1}^{n} (x_i - a)^2$$

$$\frac{d}{da} \sum (x_i - a)^2 = 0$$

$$\Rightarrow \sum (a - x_i) = 0 \;\Rightarrow na - \sum x_i = 0$$

$$\Rightarrow a = \frac{1}{n} \sum_{i=1}^{n} x_i \rightarrow \text{mean}$$

L1 objective:

$$\min_a \sum_{i=1}^{n} |x_i - a|$$

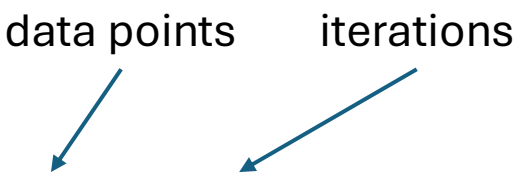
$$\frac{d}{da} \sum |x_i - a| = 0$$

$$\Rightarrow \sum \text{sign}(a - x_i) = 0$$

$$\Rightarrow \#(x_i < a) = \#(x_i > a) \rightarrow \text{median}$$

k-Means Assumptions and Limitations

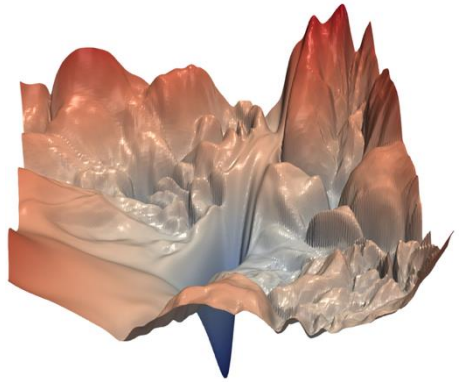
- must choose k beforehand
- sensitive to initial centroid placement
- assumes clusters are spherical
- sensitive to outliers



Advantages:

- simple and fast: $O(n \cdot k \cdot i \cdot d)$
- scales well to large datasets (MiniBatchKMeans even more so)

Cluster Initialization



k-Means depends on its starting centroids (numerical optimization over a non-convex objective: different initial centroids can lead the algorithm to different local minima)

→ Running k-means multiple times with different initializations can produce different clustering results

- Good initialization leads to better clusters, faster convergence, and more stable results
- Poor initialization can lead to suboptimal or inconsistent clustering

Practical strategies:

- Random restarts: run k-Means many times and keep the best solution
- k-means++: algorithm to spread out initial centroids (also stochastic)
- PCA-based initialization (deterministic: only one run)

Choosing the Number of Clusters

Crucial: determines whether model captures meaningful structure or produces misleading, arbitrary groupings

- Too small k → underfitting
- Too large k → overfitting / meaningless splits

Common workflow:

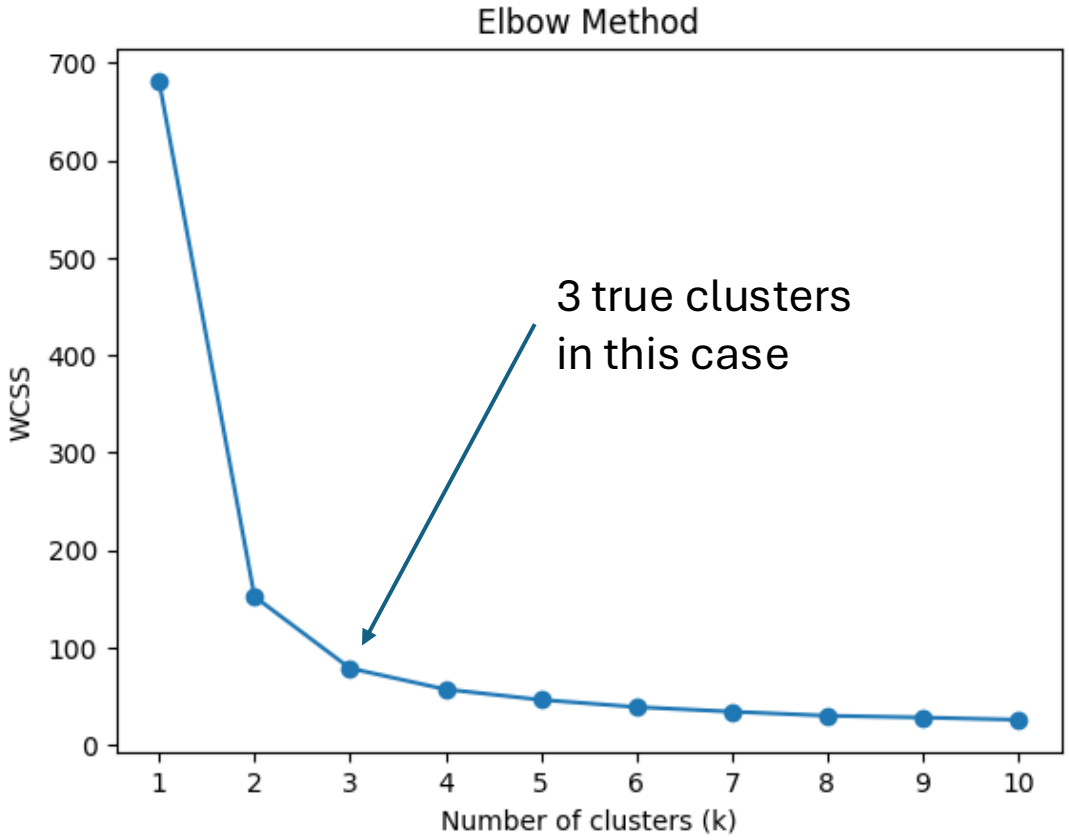
- Elbow plot for quick insight
- Compute cluster validation metrics: compute metrics for each k and choose the k with the best metric
- Visualize clusters

Elbow Plot

- Run k-means for different values of k and compute the within-cluster sum of squares (WCSS)
- Plot WCSS vs. k
- Look for a “bend” (elbow) where improvement slows down

Idea: Adding clusters always reduces error, but after a point the gain becomes small.

But elbow is often ambiguous or not visible



Cluster Validation Metrics

Since clustering is unsupervised: no single “correct” answer

Instead measure how well-separated and cohesive clusters are

Evaluation metrics include:

- Silhouette score
- Davies-Bouldin index (fast alternative)

Silhouette Score

Compares how well a point fits in its own cluster vs. how well it would fit in the next best cluster

Silhouette score = mean Silhouette coefficient over all samples

Silhouette coefficient for a sample: $(b - a)/\max(a, b)$

a: average distance from the sample to all other points in the same cluster (intra-cluster distance)

b: lowest average distance from the sample to all points in another cluster (nearest other cluster)

Score ranges from -1 to 1

- Close to 1: well-separated clusters
- Around 0: overlapping clusters
- Negative: wrong assignments

Supervised Clustering Validation

External validation: used when you already have ground-truth labels and want to evaluate how well a clustering matches the “true” grouping

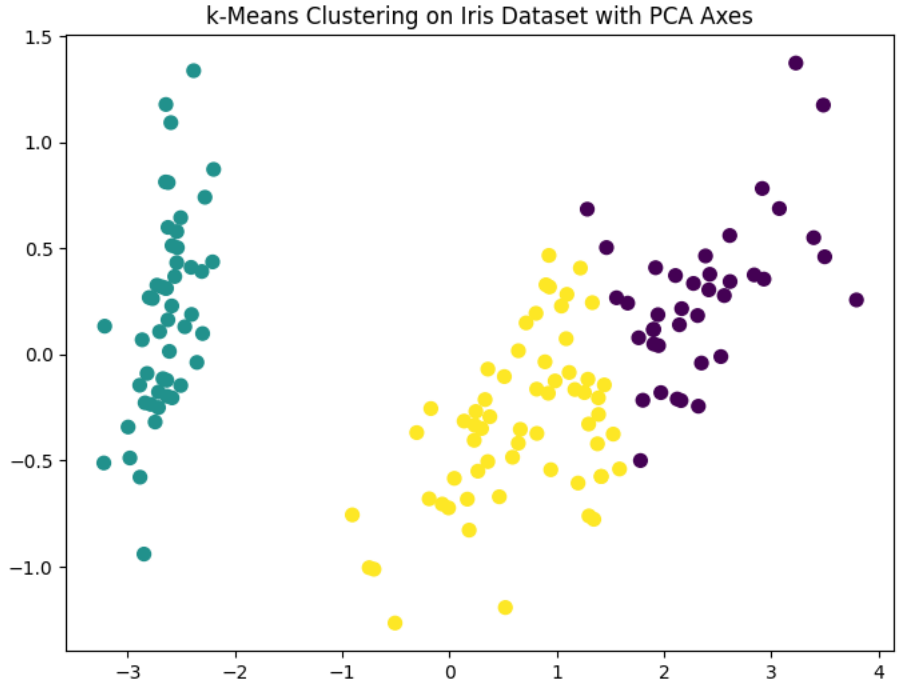
→ Evaluates correctness, not just structure

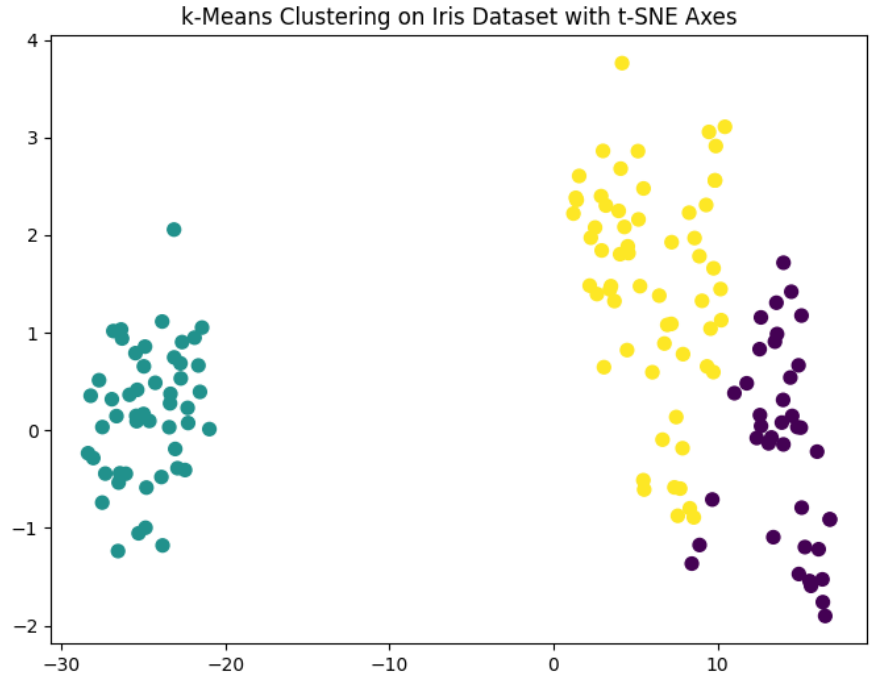
- Adjusted Rand index: counting pairs that are assigned in the same or different clusters in the predicted and true clusterings, adjusted for chance (random clustering): $\mathrm{ARI} = (\mathrm{RI} - \mathrm{Expected\_RI}) / (\max(\mathrm{RI}) - \mathrm{Expected\_RI})$
- Adjusted mutual information
- Homogeneity, completeness, and V-measure scores
  - homogeneity: each cluster contains only one class
  - completeness: all members of a class are in the same cluster
  - V-measure: harmonic mean between homogeneity and completeness

Visualizing Clusters

Even with metrics, always sanity-check:

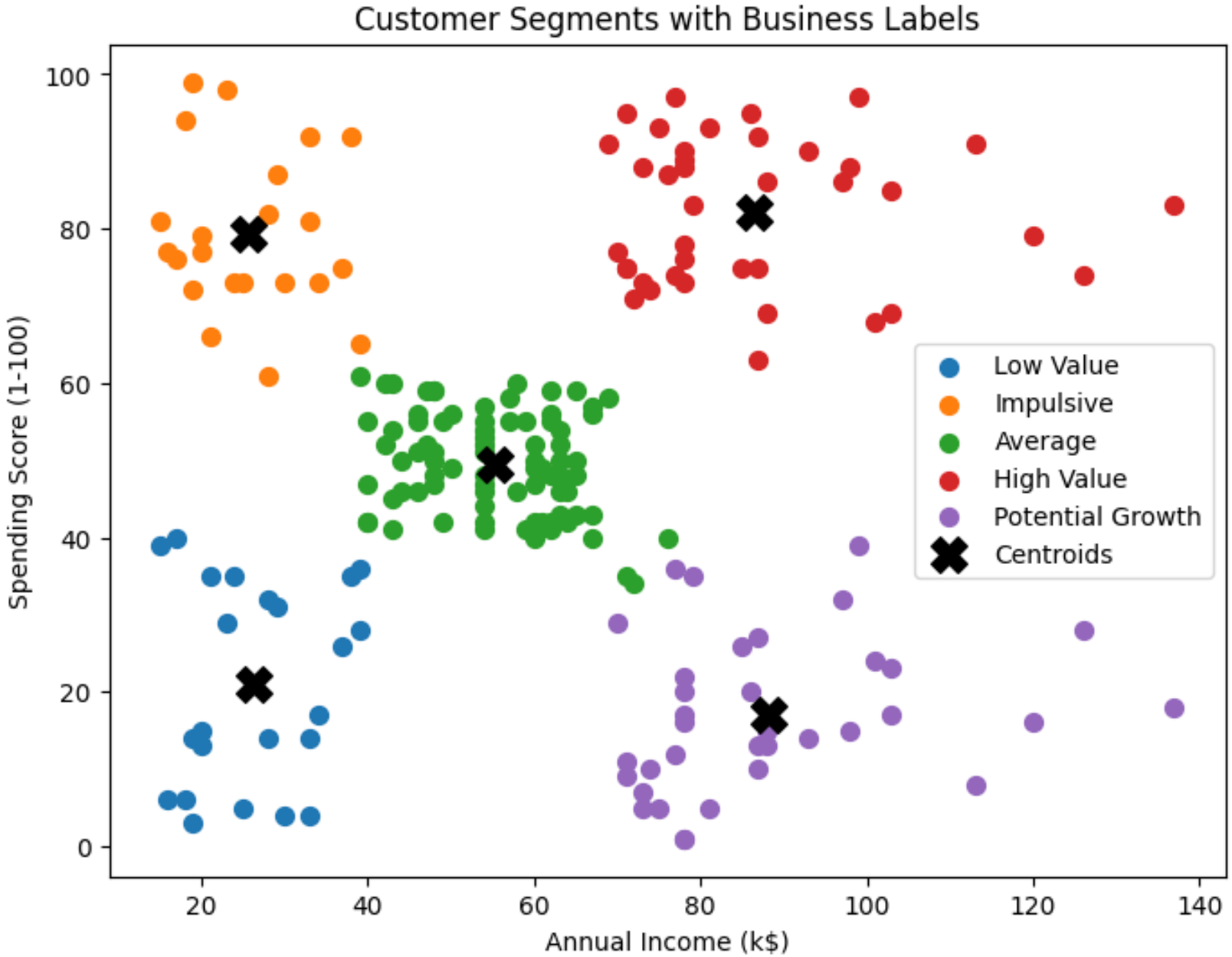
- Visualize clusters (using PCA or t-SNE for high dimensions)
- Ask: Do clusters make sense in your domain?

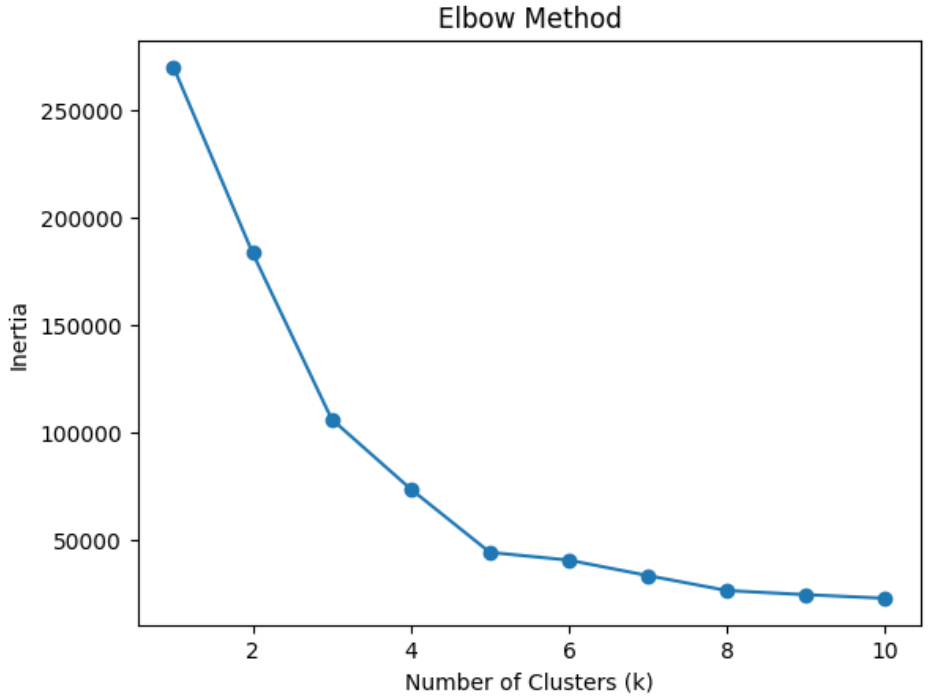




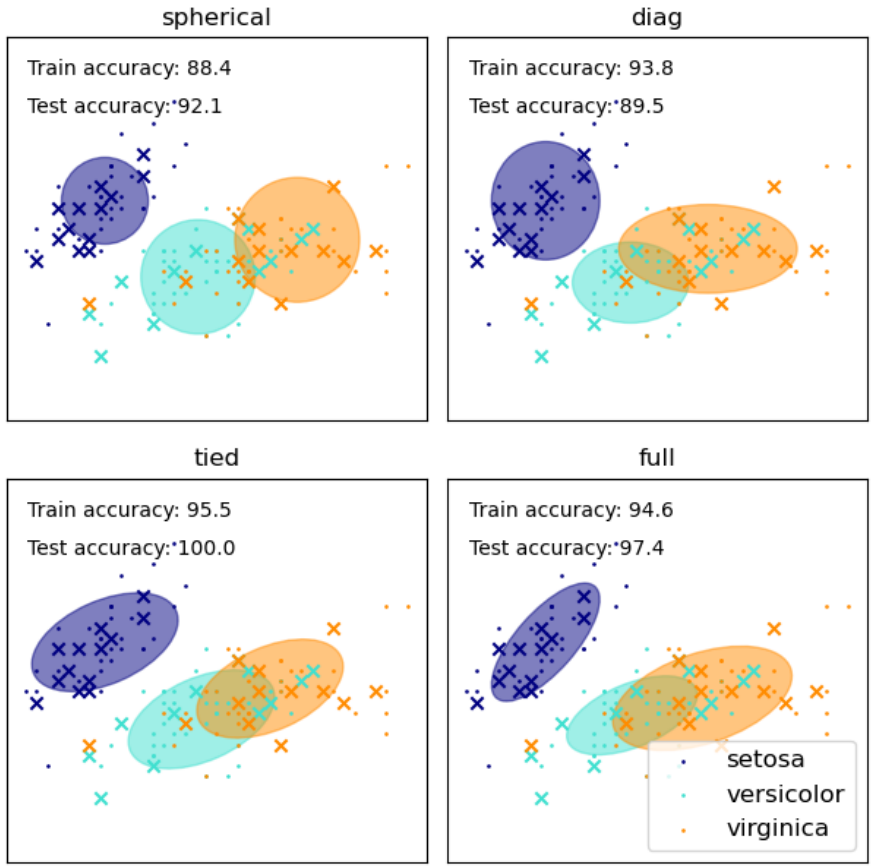
Practice Project

Customer Segmentation with k-Means on Mall Customers dataset


Customer Segments with Business Labels
Spending Score (1-100)
Annual Income (k$)
Low Value
Impulsive
Average
High Value
Potential Growth
Centroids


Elbow Method
Inertia
Number of Clusters (k)

Gaussian Mixture Models



k-Means assigns points based on Euclidean distance to the centroid

→ clusters are effectively spherical (with straight-line boundaries in 2D)

In Gaussian Mixture Models (GMM), each cluster is modeled as a multivariate Gaussian distribution (covariance matrix) → clusters can be elliptical

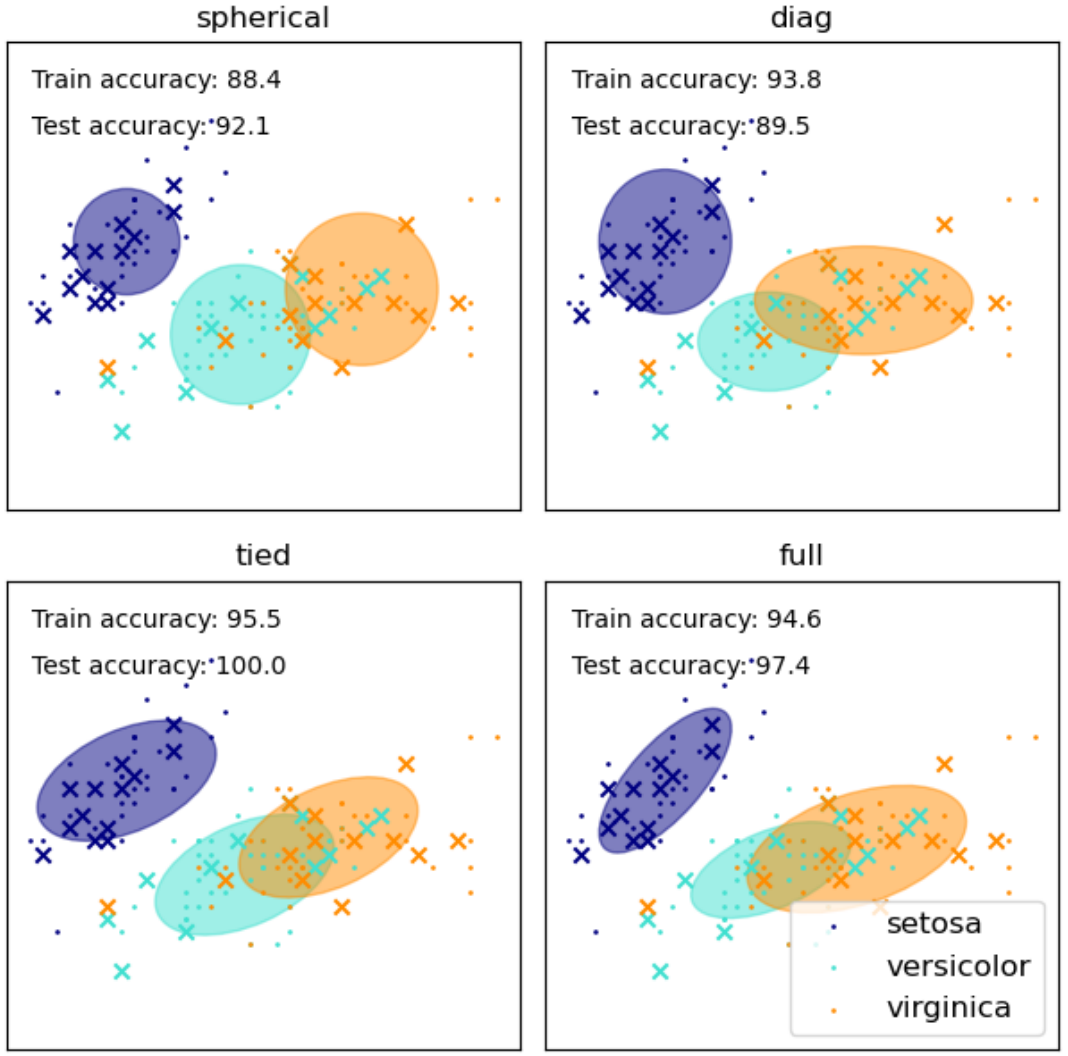
- k-Means: Group points around the nearest center
- GMM: Assume the data was generated by several Gaussian distributions and estimate them

k-Means can be interpreted as special case of Gaussian mixture model with same variance in all directions and no covariance between features

GMM Covariance Matrices

scikit learn example

- full: each component has its own general covariance matrix
- tied: all components share the same general covariance matrix
- diag: each component has its own diagonal covariance matrix
- spherical: each component has its own single variance



Hierarchical Clustering: Motivation

Builds a tree (dendrogram) of clusters
Useful for understanding nested structures

Advantages:

- No need to pre-specify number of clusters
- Captures hierarchical relationships
- Easy to visualize and interpret

But computationally expensive and sensitive to noise and outliers

Hierarchical Clustering: Key Idea

Iteratively merge or split clusters, based on a distance (similarity) measure

Two main approaches:

Agglomerative (bottom-up)

- Start with each data point as its own cluster
- Repeat:
  - Find the closest pair of clusters
  - Merge them
- Continue until:
  - One cluster remains
  - Or desired number of clusters is reached

Divisive (top-down)

- Start with all data in one cluster
- Recursively split clusters

Less commonly used (more computationally expensive)

Distance Measures and Linkage

As before, need a way to measure similarity/distance between points:

- Euclidean distance
- Manhattan distance
- Cosine similarity

→ Choice affects clustering results significantly

Cluster-to-cluster distance (linkage):

In hierarchical clustering, clusters are not summarized by a single representative point (e.g., a centroid)

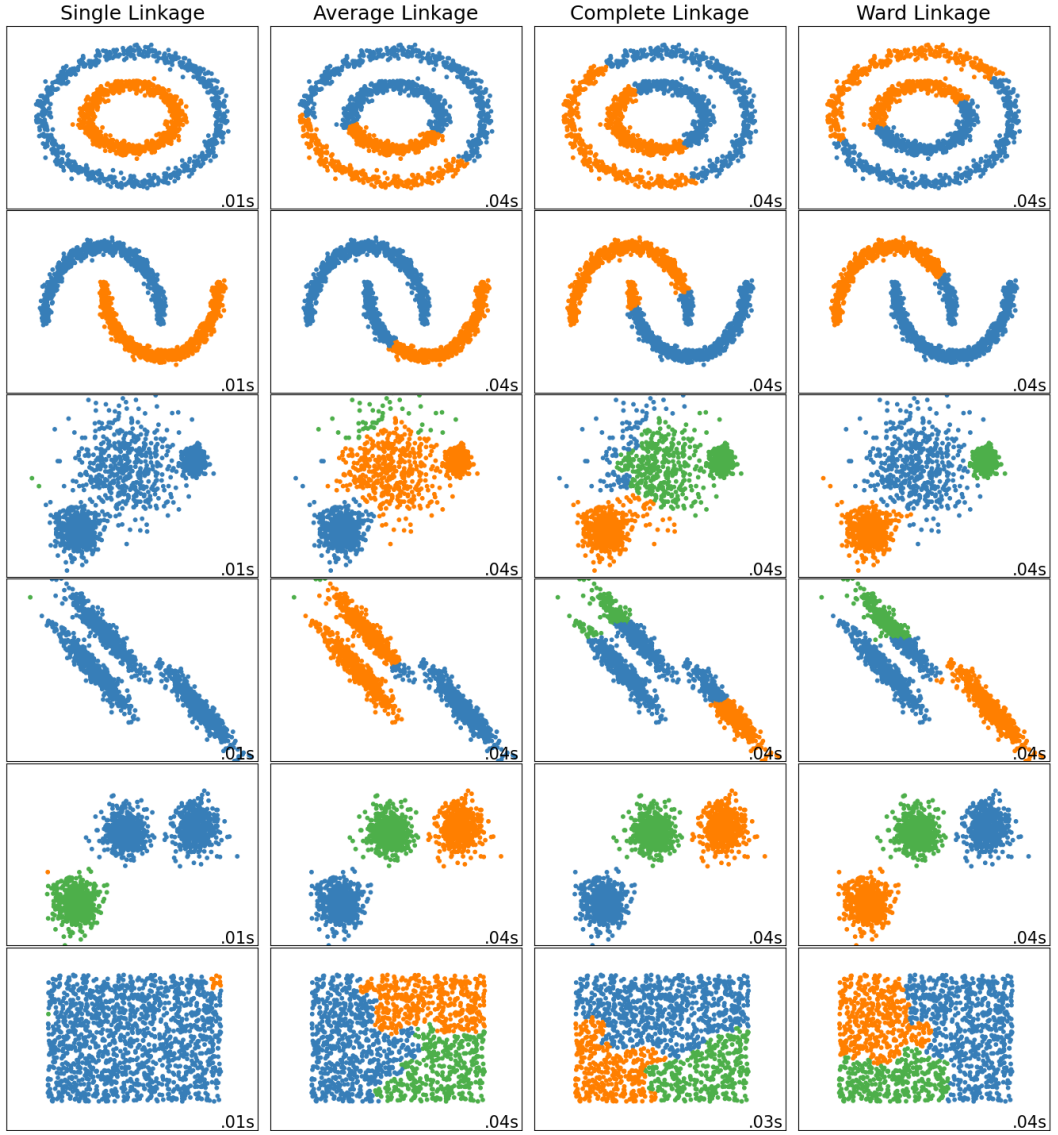
→ Need to define distance between clusters (sets of points): linkage criteria

Linkage Criteria

Different possibilities (choice strongly affects results):

- Single Linkage: Distance between closest points → can form “chains” (elongated clusters)
- Complete Linkage: Distance between farthest points → produces compact (spherical) clusters
- Average Linkage: Average pairwise distance → balanced approach
- Ward’s Method: Minimizes variance within clusters → often produces well-separated clusters

scikit learn example



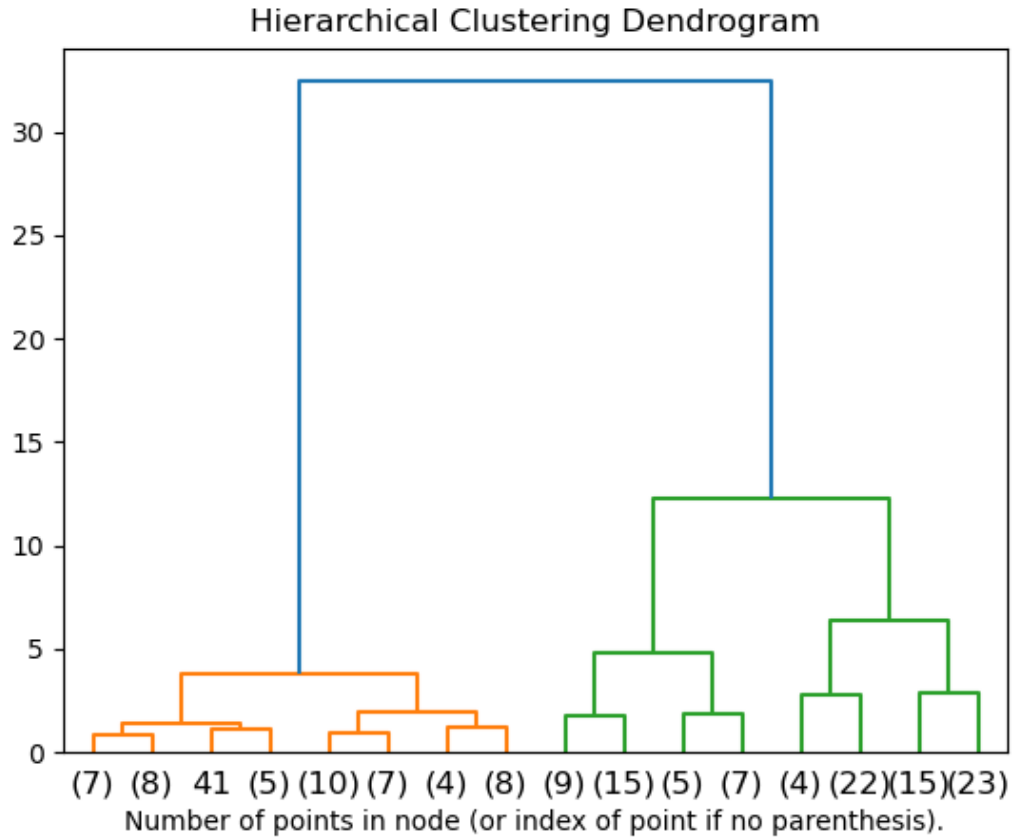
Dendrograms

Tree representation of clustering:

- Leaves = individual data points
- Internal nodes = cluster merges
- Height = distance at which clusters merge

Interpretation:

- Cut tree at a chosen height → clusters (enables flexible cluster selection)
- Large vertical gaps → natural cluster separation
- Lower merges → more similar clusters



Assign new points by distance to cluster representatives (e.g., centroids): practical heuristic, not inherent to hierarchical clustering

DBSCAN: Density-Based Clustering

DBSCAN: Density-Based Spatial Clustering of Applications with Noise

- Groups points based on dense regions
- Can detect arbitrary-shaped clusters
- Explicitly identifies noise (outliers)
- No need to specify number of clusters beforehand

Key idea: clusters are formed by dense regions of points, using two parameters:

- ε: neighborhood radius
- $\mathrm{min\_samples}$: minimum points to form a dense region

Core Points, Border Points, Noise

Define different types of points

- Core Point: have $\geq \text{min\_samples}$ points within ε → dense center
- Border Point: within ε of a core point, but not a core point itself (not dense enough) → edge of cluster
- Noise Point: not reachable from any core point → isolated point

DBSCAN Algorithm

1. Pick an unvisited point P and mark it as visited
2. Retrieve its ε-neighborhood $N_\varepsilon(P)$
3. If P is a core point:
   - Create a new cluster and add P and all points in $N_\varepsilon(P)$ to it (that are not yet assigned to any cluster)
   - For each point Q in $N_\varepsilon(P)$:
     - Mark as visited (if unvisited) and retrieve $N_\varepsilon(Q)$
     - If Q is a core core point: iteratively expand the cluster (density-reachable points)
4. Else: mark P as noise (temporary, may later become a border point)
5. Repeat until all points are processed

Cluster formation concept:

- Points are density-connected if they can be reached via a chain of core points
- Clusters grow outward from core points

Choosing ε and min_samples

Choosing ε (neighborhood radius)

- Too small → many points labeled as noise
- Too large → clusters merge
- Heuristic: use k-distance graph, look for “elbow” point

Typical values $\mathrm{min\_samples}$ (minimum points for density)

- 4 (low-dimensional data)
- Higher for high-dimensional data
- Guideline: $\mathrm{min\_samples} \geq \mathrm{dimensionality} + 1$

ε and $\mathrm{min\_samples}$ strongly affect results → tuning and data scaling matters

How to Assign New Points

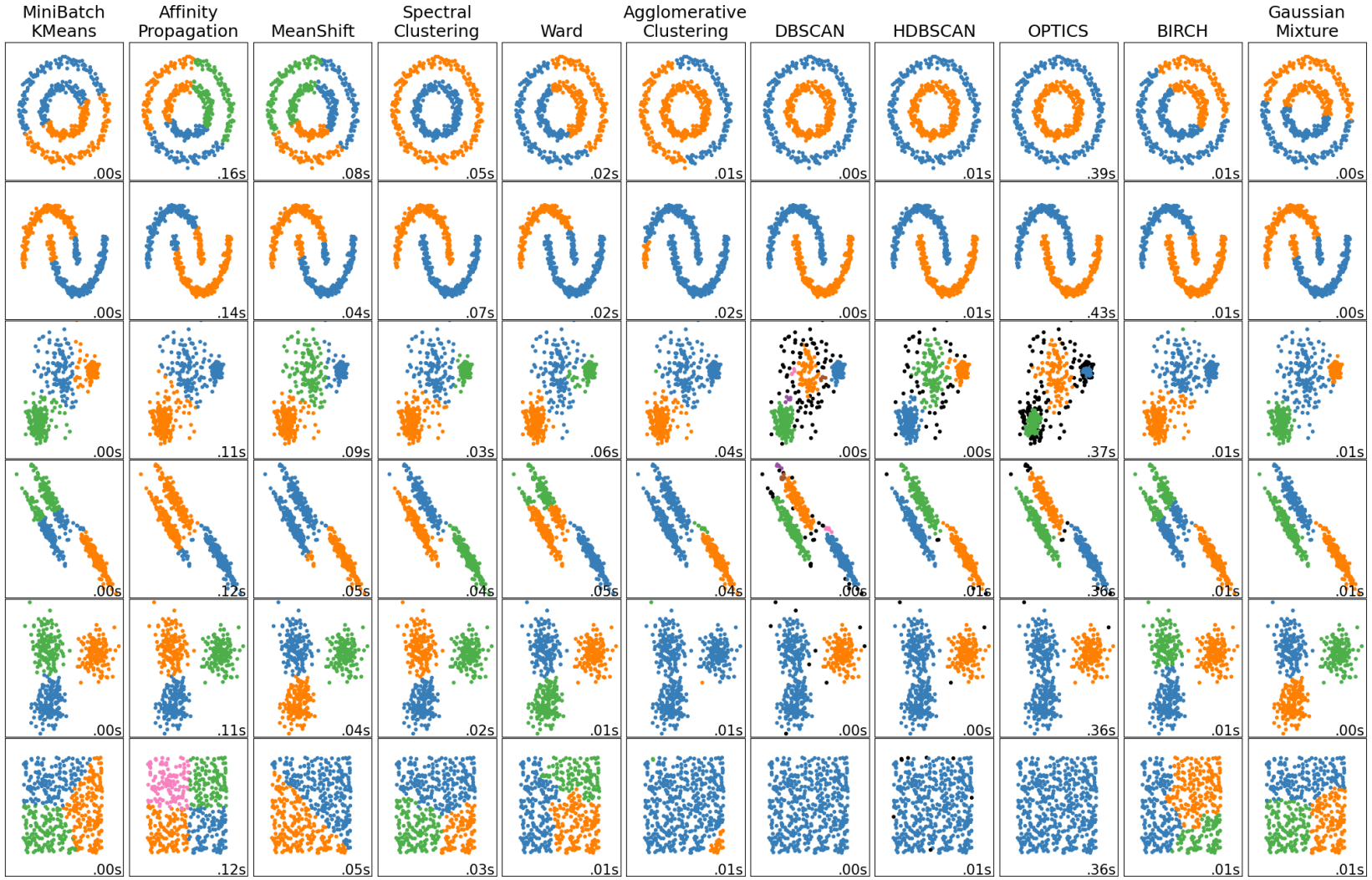
For a new point: Check its ε-neighborhood

- If within ε of a core point: Assign it to that cluster
- If not near any core point: Label as noise
- If within ε of core points from different clusters:
  - Option 1: Assign to first found (like DBSCAN would)
  - Option 2: Assign to closest core point

Adding one point can change cluster structure

→ True DBSCAN would require re-running the algorithm

Comparing Clustering Methods



Different algorithms suit different data → no single best method

Practical guidance:

- k-Means if clusters are compact and known
- Hierarchical if structure and interpretability is needed
- DBSCAN if data has noise or complex shapes

What affects results?

- k-Means: initial centroids
- Hierarchical: linkage method
- DBSCAN: ε and min_samples